What is NumPy?

NumPy stands for Numerical Python. NumPy is a Python library used for working with arrays. It is an open source project and you can use it freely. NumPy can be used to perform a wide variety of mathematical operations on arrays. It adds powerful data structures to Python that guarantee efficient calculations with arrays and matrices.

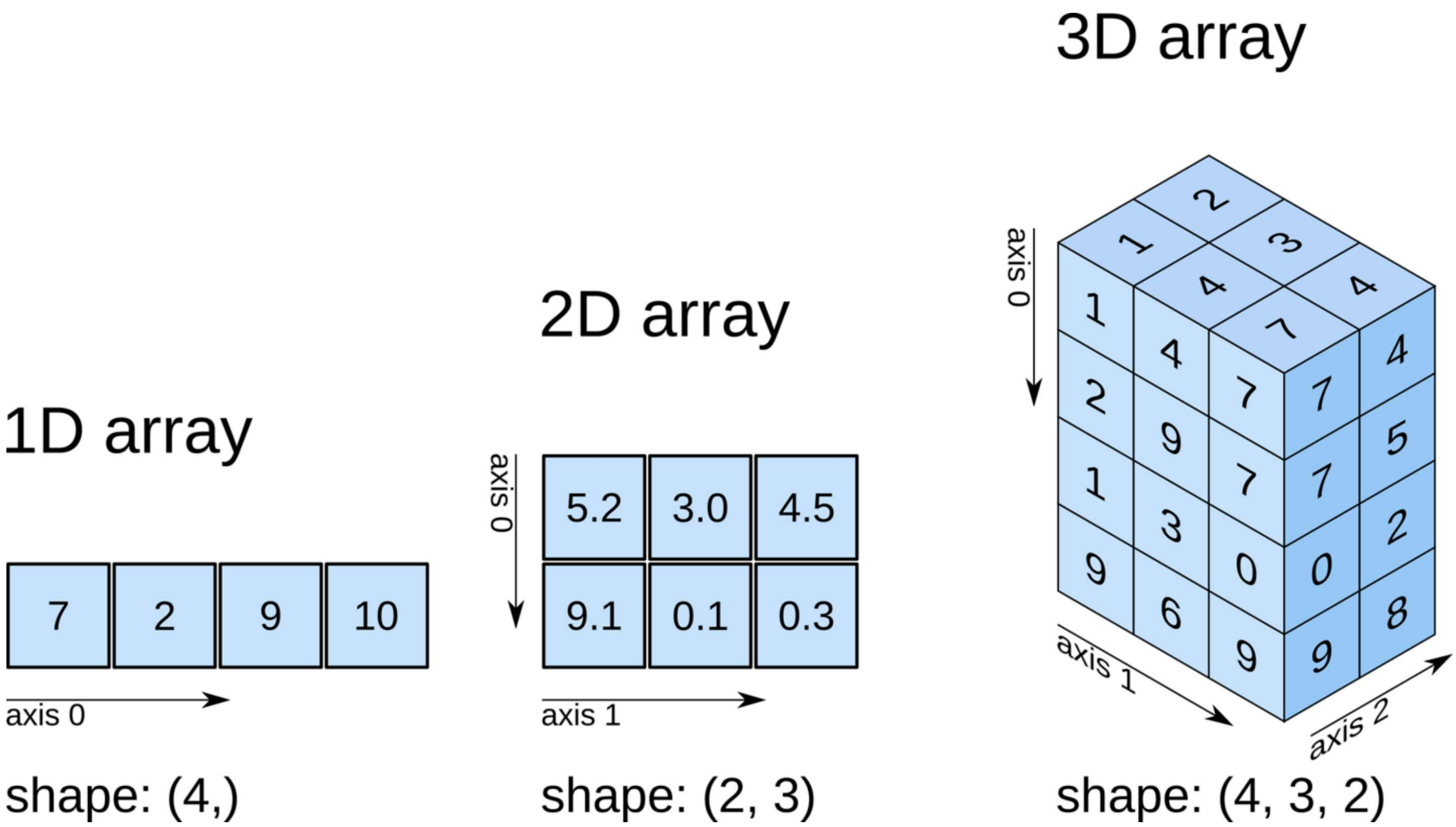
An array is a central data structure of the NumPy library. An array is a grid of values and it contains information about the raw data, how to locate an element, and how to interpret an element. The most important object defined in NumPy is an N-dimensional array type called ndarray. It describes the collection of items of the same type.

Why we use NumPy?

In Python we have lists that serve the purpose of arrays, but they are slow to process. NumPy aims to provide an array object that is up to 50x faster than traditional Python lists. The array object in NumPy is called ndarray, it provides a lot of supporting functions that make working with ndarray very easy. Arrays are very frequently used in data science, where speed and resources are very important.

What is NumPy Array?

An array is a central data structure of the NumPy library. An array is a grid of values and it contains information about the raw data, how to locate an element, and how to interpret an element. The most important object defined in NumPy is an N-dimensional array type called ndarray. It describes the collection of items of the same type.



In [ ]:

In [1]:
```
!pip install numpy
```

```
Requirement already satisfied: numpy in c:\users\dell\anaconda3\lib\site-packages (1.24.3)
```

In [2]:
```
import numpy as np
```

In [ ]:
```

```

In [9]:
```
lst = [5,4,3,3,49,5]
lst
```

Out[9]:
```
[5, 4, 3, 3, 49, 5]
```

In [10]:
```
type(lst)
```

Out[10]:
```
list
```

In [11]:
```
arr = np.array(lst)
print(arr)
```

```
[ 5  4  3  3 49  5]
```

In [12]:
```
arr = np.array([5,4,3,3,49,5])
print(arr)
```

```
[ 5  4  3  3 49  5]
```

In [13]:
```
arr
```

Out[13]:
```
array([ 5,  4,  3,  3, 49,  5])
```

In [14]:
```
type(arr)
```

Out[14]:
```
numpy.ndarray
```

In [ ]:
```

```

Difference between Numpy Array and Python List

Type *Markdown* and LaTeX: α^2

```
Data Type Storage
1.Homogeneous
2.Heterogeneous

Numerical Operations
1.Vectorized
2.Iterative

Performance and Speed
1.High Speed and Less Memory
2.Low Speed and More Memory
```

Difference 1

In [ ]:

In [2]:
```
lst = [1,2,3.6,4,1.6,4,'a', True, 'deepak']
lst
```

Out[2]:
```
[1, 2, 3.6, 4, 1.6, 4, 'a', True, 'deepak']
```

In [ ]:

In [5]:
```
lst = [5,4,3,3,49,5]
lst
```

Out[5]:
```
[5, 4, 3, 3, 49, 5]
```

In [ ]:

In [9]:
```
import numpy as np
```

In [11]:
```
arr = np.array([1,2,3,4,5])
arr = arr * 5
```

In [12]:
```
print(arr)
```

```
[ 5 10 15 20 25]
```

In [ ]:

In [14]:
```
arr = ([1,2,3,4.1, 'a'])
arr
```

Out[14]:
```
[1, 2, 3, 4.1, 'a']
```

In [ ]:

In [15]:
```
arr = np.array([1,2,3,4.1, 'a'])
arr
```

Out[15]:
```
array(['1', '2', '3', '4.1', 'a'], dtype='<U32')
```

In [ ]:

In [17]:
```
arr.dtype
```

Out[17]:
```
dtype('<U32')
```

In [ ]:

In [18]:
```
arr = np.array([2,4,6, 'f'])
arr
```

Out[18]:
```
array(['2', '4', '6', 'f'], dtype='<U11')
```

In [ ]:
```

```

Difference 2

In [21]:
```
arr = np.array([4,5,6,7,8])
arr = arr * 6
arr
```

Out[21]:
```
array([24, 30, 36, 42, 48])
```

In [ ]:
```

```

In [23]:
```
arr = np.array([4,5,6,7,8]) * 6
arr
```

Out[23]:
```
array([24, 30, 36, 42, 48])
```

In [ ]:
```

```

In [25]:
```
lst = [4,5,6,7,8] * 6
lst
print(lst)
```

```
[4, 5, 6, 7, 8, 4, 5, 6, 7, 8, 4, 5, 6, 7, 8, 4, 5, 6, 7, 8, 4, 5, 6, 7, 8, 4, 5, 6, 7, 8]
```

In [ ]:
```

```

In [45]:
```
lst1 = [4,5,6,7,8]
lst
```

Out[45]:
```
[4, 5, 6, 7, 8]
```

In [ ]:
```

```

In [47]:
```
lst1 = [4,5,6,7,8]

l = []

for i in lst1:
    l.append(i*6)

l
```

Out[47]:
```
[24, 30, 36, 42, 48]
```

In [ ]:
```

```

Difference 3

In [7]:
```
list(range(2,22))
```

Out[7]:
```
[2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21]
```

In [67]:
```
[i**2 for i in range(2,22)]
```

Out[67]:
```
[4,
 9,
 16,
 25,
 36,
 49,
 64,
 81,
 100,
 121,
 144,
 169,
 196,
 225,
 256,
 289,
 324,
 361,
 400,
 441]
```

In [13]:
```
%timeit [i**2 for i in range (1,500)]
```

```
25.8 µs ± 231 ns per loop (mean ± std. dev. of 7 runs, 10,000 loops each)
```

In [ ]:

Creating 1D, 2D and 3D Array

In [ ]:

In [ ]:

In [ ]:

In [2]:
```
import numpy as np
```

In [3]:
```
arr1 = np.array([5,6,7,8])
arr1
```

Out[3]:
```
array([5, 6, 7, 8])
```

In [4]:
```
arr1.ndim
```

Out[4]:
```
1
```

In [5]:
```
arr1.shape
```

Out[5]:
```
(4,)
```

In [ ]:

In [6]:
```
arr2 = np.array([[5,6,7,8]])
arr2
```

Out[6]:
```
array([[5, 6, 7, 8]])
```

In [7]:
```
arr2.ndim
```

Out[7]:
```
2
```

In [8]:
```
arr2.shape
```

Out[8]:
```
(1, 4)
```

In [ ]:

In [9]:
```
arr2 = np.array([[5,6],[7,8]])
arr2
```

Out[9]:
```
array([[5, 6],
       [7, 8]])
```

In [ ]:
```

```

In [10]:
```
arr2.ndim
```

Out[10]:
```
2
```

In [11]:
```
arr2.shape
```

Out[11]:
```
(2, 2)
```

In [ ]:
```

```

In [12]:
```
arr2 = np.array([[5,6,7,8],[9,10,11,12]])
arr2
```

Out[12]:
```
array([[ 5,  6,  7,  8],
       [ 9, 10, 11, 12]])
```

In [13]:
```
arr2.ndim
```

Out[13]:
```
2
```

In [14]:
```
arr2.shape
```

Out[14]:
```
(2, 4)
```

In [15]:
```
arr2 = np.array([[5,6,7],[8,9,10]])
arr2
```

Out[15]:
```
array([[ 5,  6,  7],
       [ 8,  9, 10]])
```

In [16]:
```
arr2.ndim
```

Out[16]:
```
2
```

In [17]:
```
arr2.shape
```

Out[17]:
```
(2, 3)
```

In [ ]:
```

```

In [18]:
```
arr2 = np.array([[5,6,7],[8,9,10], [10,11,12]])
arr2
```

Out[18]:
```
array([[ 5,  6,  7],
       [ 8,  9, 10],
       [10, 11, 12]])
```

In [19]:
```
arr2.ndim
```

Out[19]:
```
2
```

In [20]:
```
arr2.shape
```

Out[20]:
```
(3, 3)
```

In [ ]:
```

```

In [28]:
```
arr3 = np.array([[[5,6,7,8,9]]])
arr3
```

Out[28]:
```
array([[[5, 6, 7, 8, 9]]])
```

In [29]:
```
arr3.ndim
```

Out[29]:
```
3
```

In [30]:
```
arr3.shape
```

Out[30]:
```
(1, 1, 5)
```

In [ ]:
```

```

In [34]:
```
arr3 = np.array([[[5,6,7,8,9,],[10,11,12,13,14],[15,16,17,18,19]]])
arr3
```

Out[34]:
```
array([[[ 5,  6,  7,  8,  9],
        [10, 11, 12, 13, 14],
        [15, 16, 17, 18, 19]]])
```

In [35]:
```
arr3.ndim
```

Out[35]:
```
3
```

In [36]:
```
arr3.shape
```

Out[36]:
```
(1, 3, 5)
```

In [ ]:
```

```

In [37]:
```
arr20 = np.array([5,6,7,8,9], ndmin=20)
arr20
```

Out[37]:
```
array([[[[[[[[[[[[[[[[[[[[5, 6, 7, 8, 9]]]]]]]]]]]]]]]]]]]])
```

In [38]:
```
arr20.ndim
```

Out[38]:
```
20
```

In [40]:
```
arr20.shape
```

Out[40]:
```
(1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 5)
```

In [43]:
```
arr60 = np.array([5,6,7,8,9], ndmin=60)
arr60
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[43], line 1
----> 1 arr60 = np.array([5,6,7,8,9], ndmin=60)
      2 arr60

ValueError: ndmin bigger than allowable number of dimensions NPY_MAXDIMS (=32)
```

In [45]:
```
arr32 = np.array([5,6,7,8,9], ndmin=32)
arr32
```

Out[45]:
```
array([[[[[[[[[[[[[[[[[[[[[[[[[[[[[[[[5, 6,
                                       7, 8,
                                       9]]]]]]]]]]]]]]]]]]]]]]]]]]]]])
```

In [ ]:
```

```

Special Numpy Array

In [47]:
```
import numpy as np
```

In [50]:
```
np.zeros(6)
```

Out[50]:
```
array([0., 0., 0., 0., 0., 0.])
```

In [51]:
```
np.zeros(6, dtype='int')
```

Out[51]:
```
array([0, 0, 0, 0, 0, 0])
```

In [52]:
```
np.zeros([4,6], dtype= 'int')
```

Out[52]:
```
array([[0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0]])
```

In [53]:
```
np.ones(6)
```

Out[53]:
```
array([1., 1., 1., 1., 1., 1.])
```

In [55]:
```
np.ones((5,5), dtype='str')
```

Out[55]:
```
array([['1', '1', '1', '1', '1'],
       ['1', '1', '1', '1', '1'],
       ['1', '1', '1', '1', '1'],
       ['1', '1', '1', '1', '1'],
       ['1', '1', '1', '1', '1']], dtype='<U1')
```

In [56]:
```
np.eye(9)
```

Out[56]:
```
array([[1., 0., 0., 0., 0., 0., 0., 0., 0.],
       [0., 1., 0., 0., 0., 0., 0., 0., 0.],
       [0., 0., 1., 0., 0., 0., 0., 0., 0.],
       [0., 0., 0., 1., 0., 0., 0., 0., 0.],
       [0., 0., 0., 0., 1., 0., 0., 0., 0.],
       [0., 0., 0., 0., 0., 1., 0., 0., 0.],
       [0., 0., 0., 0., 0., 0., 1., 0., 0.],
       [0., 0., 0., 0., 0., 0., 0., 1., 0.],
       [0., 0., 0., 0., 0., 0., 0., 0., 1.]])
```

In [57]:
```
np.full([4,4], 8.88)
```

Out[57]:
```
array([[8.88, 8.88, 8.88, 8.88],
       [8.88, 8.88, 8.88, 8.88],
       [8.88, 8.88, 8.88, 8.88],
       [8.88, 8.88, 8.88, 8.88]])
```

In [58]:
```
np.full([5,5], 'Deepak')
```

Out[58]:
```
array([['Deepak', 'Deepak', 'Deepak', 'Deepak', 'Deepak'],
       ['Deepak', 'Deepak', 'Deepak', 'Deepak', 'Deepak'],
       ['Deepak', 'Deepak', 'Deepak', 'Deepak', 'Deepak'],
       ['Deepak', 'Deepak', 'Deepak', 'Deepak', 'Deepak'],
       ['Deepak', 'Deepak', 'Deepak', 'Deepak', 'Deepak']], dtype='<U6')
```

In [60]:
```
np.full([4,4], 2)
```

Out[60]:
```
array([[2, 2, 2, 2],
       [2, 2, 2, 2],
       [2, 2, 2, 2],
       [2, 2, 2, 2]])
```

In [62]:
```
np.arange(1,201)
```

Out[62]:
```
array([  1,   2,   3,   4,   5,   6,   7,   8,   9,  10,  11,  12,  13,
        14,  15,  16,  17,  18,  19,  20,  21,  22,  23,  24,  25,  26,
        27,  28,  29,  30,  31,  32,  33,  34,  35,  36,  37,  38,  39,
        40,  41,  42,  43,  44,  45,  46,  47,  48,  49,  50,  51,  52,
        53,  54,  55,  56,  57,  58,  59,  60,  61,  62,  63,  64,  65,
        66,  67,  68,  69,  70,  71,  72,  73,  74,  75,  76,  77,  78,
        79,  80,  81,  82,  83,  84,  85,  86,  87,  88,  89,  90,  91,
        92,  93,  94,  95,  96,  97,  98,  99, 100, 101, 102, 103, 104,
       105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117,
       118, 119, 120, 121, 122, 123, 124, 125, 126, 127, 128, 129, 130,
       131, 132, 133, 134, 135, 136, 137, 138, 139, 140, 141, 142, 143,
       144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154, 155, 156,
       157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 169,
       170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182,
       183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195,
       196, 197, 198, 199, 200])
```

In [63]:
```
np.arange(0,201,6)
```

Out[63]:
```
array([  0,   6,  12,  18,  24,  30,  36,  42,  48,  54,  60,  66,  72,
        78,  84,  90,  96, 102, 108, 114, 120, 126, 132, 138, 144, 150,
       156, 162, 168, 174, 180, 186, 192, 198])
```

In [64]:
```
np.linspace(1, 20, 20)
```

Out[64]:
```
array([ 1.,  2.,  3.,  4.,  5.,  6.,  7.,  8.,  9., 10., 11., 12., 13.,
       14., 15., 16., 17., 18., 19., 20.])
```

In [65]:
```
np.linspace(1, 20, 40)
```

Out[65]:
```
array([ 1.        ,  1.48717949,  1.97435897,  2.46153846,  2.94871795,
        3.43589744,  3.92307692,  4.41025641,  4.8974359 ,  5.38461538,
        5.87179487,  6.35897436,  6.84615385,  7.33333333,  7.82051282,
        8.30769231,  8.79487179,  9.28205128,  9.76923077, 10.25641026,
       10.74358974, 11.23076923, 11.71794872, 12.20512821, 12.69230769,
       13.17948718, 13.66666667, 14.15384615, 14.64102564, 15.12820513,
       15.61538462, 16.1025641 , 16.58974359, 17.07692308, 17.56410256,
       18.05128205, 18.53846154, 19.02564103, 19.51282051, 20.        ])
```

In [ ]:
```

```

Random Numbers Generator

In [2]:
```
from numpy import random
```

In [22]:
```
arr = random.randint(0,1000)
arr
```

Out[22]:
```
776
```

In [24]:
```
arr =random.randint(200,size=(10))
arr
```

Out[24]:
```
array([ 77,  22,  56, 192, 185,  35, 108, 102, 136, 156])
```

In [25]:
```
arr = random.randint(200, size=(3,4))
arr
```

Out[25]:
```
array([[138, 133, 172, 145],
       [106,  49, 197,  21],
       [169, 168,  24,  14]])
```

In [9]:
```
random.rand(4)
```

Out[9]:
```
array([0.02944629, 0.22098414, 0.72617971, 0.20359523])
```

In [27]:
```
random.rand(3,4)
```

Out[27]:
```
array([[0.27368468, 0.56242981, 0.56946809, 0.19633341],
       [0.57993605, 0.28295022, 0.44898379, 0.38957016],
       [0.7635015 , 0.91759255, 0.86860249, 0.98272892]])
```

In [ ]:
```

```

In [29]:
```
random.choice([7,8,2,3,5], size=1)
```

Out[29]:
```
array([8])
```

In [31]:
```
random.choice([7,8,2,3,5], size=4)
```

Out[31]:
```
array([3, 7, 8, 8])
```

In [32]:
```
random.choice([7,8,2,3,5], size=10)
```

Out[32]:
```
array([2, 7, 5, 8, 2, 7, 2, 5, 5, 2])
```

In [34]:
```
random.choice([7,8,2,3,5], size=100)
```

Out[34]:
```
array([7, 3, 8, 2, 5, 2, 7, 3, 7, 7, 2, 3, 7, 2, 3, 5, 5, 3, 3, 3, 8, 7,
       7, 3, 5, 5, 3, 7, 5, 2, 2, 8, 5, 8, 8, 2, 3, 8, 3, 8, 3, 8, 3, 7,
       8, 2, 2, 5, 3, 8, 3, 2, 5, 8, 2, 3, 2, 2, 2, 3, 7, 2, 7, 3, 8, 3,
       7, 2, 8, 2, 8, 8, 3, 5, 3, 5, 5, 5, 3, 7, 3, 7, 3, 3, 8, 5, 2, 7,
       3, 3, 2, 5, 8, 3, 5, 5, 7, 5, 3, 8])
```

In [ ]:

In [44]:
```
random.choice([7,8,2,3,5], p=[0.0, 0.1, 0.0, 0.9],  size=100)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[44], line 1
----> 1 random.choice([7,8,2,3,5], p=[0.0, 0.1, 0.0, 0.9],  size=100)

File mtrand.pyx:951, in numpy.random.mtrand.RandomState.choice()

ValueError: 'a' and 'p' must have same size
```

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

Data Type Conversion

In [45]:
```
arr = np.array([9,8,7,6,5])
print(arr)
print(arr.dtype)
```

```
[9 8 7 6 5]
int32
```

In [46]:
```
arr = np.array([9,8,7,6,5], dtype='float')
print(arr)
print(arr.dtype)
```

```
[9. 8. 7. 6. 5.]
float64
```

In [47]:
```
arr = np.array([9,8,7,6,5], dtype='f')
print(arr)
print(arr.dtype)
```

```
[9. 8. 7. 6. 5.]
float32
```

In [51]:
```
arr = np.array([9,8,7,6,5], dtype='O')
print(arr)
print(arr.dtype)
```

```
[9 8 7 6 5]
object
```

In [52]:
```
arr = np.array([9,8,7,6,5], dtype='str')
print(arr)
print(arr.dtype)
```

```
['9' '8' '7' '6' '5']
<U1
```

In [53]:
```
arr = np.array([9,8,7.5,6.3,5], dtype='str')
print(arr)
print(arr.dtype)
```

```
['9' '8' '7.5' '6.3' '5']
<U3
```

In [55]:
```
arr = np.array([9,8,7,6,5, 'f', 'g' ], dtype='str')
print(arr)
print(arr.dtype)
```

```
['9' '8' '7' '6' '5' 'f' 'g']
<U1
```

In [56]:
```
arr = np.array(['f', 'g'], dtype='int')
print(arr)
print(arr.dtype)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[56], line 1
----> 1 arr = np.array(['f', 'g'], dtype='int')
      2 print(arr)
      3 print(arr.dtype)

ValueError: invalid literal for int() with base 10: 'f'
```

In [57]:
```
arr = np.array(['4', '5'], dtype='int')
print(arr)
print(arr.dtype)
```

```
[4 5]
int32
```

Memory Management

Name	Type	Range
std::int8_t	1 byte signed	-128 to 127
std::uint8_t	1 byte unsigned	0 to 255
std::int16_t	2 byte signed	-32,768 to 32,767
std::uint16_t	2 byte unsigned	0 to 65,535
std::int32_t	4 byte signed	-2,147,483,648 to 2,147,483,647
std::uint32_t	4 byte unsigned	0 to 4,294,967,295
std::int64_t	8 byte signed	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
std::uint64_t	8 byte unsigned	0 to 18,446,744,073,709,551,615

In [59]:
```
arr = np.array([9,8,7,6,5])
print(arr)
print(arr.dtype)
```

```
[9 8 7 6 5]
int32
```

In [60]:
```
arr = np.array([9,8,7,6,5],  np.int8)
print(arr)
print(arr.dtype)
```

```
[9 8 7 6 5]
int8
```

In [62]:
```
arr = np.array([9,8,745,6,5],  np.int8)
print(arr)
print(arr.dtype)
```

```
[  9   8 -23   6   5]
int8

C:\Users\Dell\AppData\Local\Temp\ipykernel_20320\2096504961.py:1: DeprecationWarning: NumPy will stop allowing conver
sion of out-of-bound Python integers to integer arrays.  The conversion of 745 to int8 will fail in the future.
For the old behavior, usually:
    np.array(value).astype(dtype)`
will give the desired result (the cast overflows).
  arr = np.array([9,8,745,6,5],  np.int8)
```

In [63]:
```
arr = np.array([9,8,1745,6,5],  np.int16)
print(arr)
print(arr.dtype)
```

```
[   9    8 1745    6    5]
int16
```

In [67]:
```
arr = np.array([9,8,-37745,6,5],  np.int16)
print(arr)
print(arr.dtype)
```

```
[    9     8 27791     6     5]
int16

C:\Users\Dell\AppData\Local\Temp\ipykernel_20320\2747760950.py:1: DeprecationWarning: NumPy will stop allowing conver
sion of out-of-bound Python integers to integer arrays.  The conversion of -37745 to int16 will fail in the future.
For the old behavior, usually:
    np.array(value).astype(dtype)`
will give the desired result (the cast overflows).
  arr = np.array([9,8,-37745,6,5],  np.int16)
```

In [ ]:
```
```

Arithmatics Operation

In [68]:
```
arr1 = np.array([4,5,6,7,8])
arr2 = np.array([8,10,12,14,16])

arr1 + arr2
```

Out[68]:
```
array([12, 15, 18, 21, 24])
```

In [69]:
```
arr1 = np.array([4,5,6,7,8])
arr2 = np.array([8,10,12,14,16])

arr1 - arr2
```

Out[69]:
```
array([-4, -5, -6, -7, -8])
```

In [70]:
```
arr1 = np.array([4,5,6,7,8])
arr2 = np.array([8,10,12,14,16])

arr1 * arr2
```

Out[70]:
```
array([ 32,  50,  72,  98, 128])
```

In [71]:
```
arr1 = np.array([4,5,6,7,8])
arr2 = np.array([8,10,12,14,16])

arr1 / arr2
```

Out[71]:
```
array([0.5, 0.5, 0.5, 0.5, 0.5])
```

In [72]:
```
arr1 = np.array([4,5,6,7,8])
arr2 = np.array([8,10,12,14,16])

arr1 // arr2
```

Out[72]:
```
array([0, 0, 0, 0, 0])
```

In [73]:
```
arr1 = np.array([4,5,6,7,8])
arr2 = np.array([8,10,12,14,16])

arr1 ** arr2
```

Out[73]:
```
array([      65536,     9765625, -2118184960,  -381759919,           0])
```

In [ ]:
```

```

Arithmatic Operation Using Functions

In [74]:
```
arr1 = np.array([4,5,6,7,8])
arr2 = np.array([8,10,12,14,16])

np.add(arr1, arr2)
```

Out[74]:
```
array([12, 15, 18, 21, 24])
```

In [75]:
```
arr1 = np.array([4,5,6,7,8])
arr2 = np.array([8,10,12,14,16])

np.subtract(arr1, arr2)
```

Out[75]:
```
array([-4, -5, -6, -7, -8])
```

In [76]:
```
arr1 = np.array([4,5,6,7,8])
arr2 = np.array([8,10,12,14,16])

np.multiply(arr1, arr2)
```

Out[76]:
```
array([ 32,  50,  72,  98, 128])
```

In [77]:
```
arr1 = np.array([4,5,6,7,8])
arr2 = np.array([8,10,12,14,16])

np.divide(arr1, arr2)
```

Out[77]:
```
array([0.5, 0.5, 0.5, 0.5, 0.5])
```

In [78]:
```
arr1 = np.array([4,5,6,7,8])
arr2 = np.array([8,10,12,14,16])

np.floor_divide(arr1, arr2)
```

Out[78]:
```
array([0, 0, 0, 0, 0])
```

In [79]:
```
arr1 = np.array([4,5,6,7,8])
arr2 = np.array([8,10,12,14,16])

np.mod(arr1, arr2)
```

Out[79]:
```
array([4, 5, 6, 7, 8])
```

In [80]:
```
arr1 = np.array([4,5,6,7,8])
arr2 = np.array([8,10,12,14,16])

np.power(arr1, arr2)
```

Out[80]:
```
array([      65536,     9765625, -2118184960,  -381759919,           0])
```

In [ ]:
```

```

Statistical Functions

In [82]:
```
np.array([5,3,2,98,0,6,5,8383,937,25242,85,73,42])
```

Out[82]:
```
array([    5,     3,     2,    98,     0,     6,     5,  8383,   937,
       25242,    85,    73,    42])
```

In [83]:
```
np.sum([5,3,2,98,0,6,5,8383,937,25242,85,73,42])
```

Out[83]:
```
34881
```

In [84]:
```
len([5,3,2,98,0,6,5,8383,937,25242,85,73,42])
```

Out[84]:
```
13
```

In [85]:
```
34881/13
```

Out[85]:
```
2683.153846153846
```

In [86]:
```
np.mean([5,3,2,98,0,6,5,8383,937,25242,85,73,42])
```

Out[86]:
```
2683.153846153846
```

In [89]:
```
np.median([5,3,2,98,0,6,5,8383,937,25242,85,73,42])
```

Out[89]:
```
42.0
```

In [90]:
```
np.sort([5,3,2,98,0,6,5,8383,937,25242,85,73,42])
```

Out[90]:
```
array([    0,     2,     3,     5,     5,     6,    42,    73,    85,
          98,   937,  8383, 25242])
```

In [91]:
```
np.sort([5,3,2,98,0,6,5,8383,937,25242,85,73,42])[::-1]
```

Out[91]:
```
array([25242,  8383,   937,    98,    85,    73,    42,     6,     5,
           5,     3,     2,     0])
```

In [92]:
```
lst = [5,3,2,98,0,6,5,8383,937,25242,85,73,42]
lst
```

Out[92]:
```
[5, 3, 2, 98, 0, 6, 5, 8383, 937, 25242, 85, 73, 42]
```

In [93]:
```
lst.sort()
lst
```

Out[93]: [0, 2, 3, 5, 5, 6, 42, 73, 85, 98, 937, 8383, 25242]

In [94]:
```
lst.sort(reverse=True)
lst
```

Out[94]: [25242, 8383, 937, 98, 85, 73, 42, 6, 5, 5, 3, 2, 0]

In [95]:
```
np.std([5,3,2,98,0,6,5,8383,937,25242,85,73,42])
```

Out[95]: 6876.627939803432

In [96]:
```
np.var([5,3,2,98,0,6,5,8383,937,25242,85,73,42])
```

Out[96]: 47288011.8224852

In [97]:
```
6876.627939803432 ** 2
```

Out[97]: 47288011.82248519

In [ ]:
```

```

Manual way to Calculate your Variance and Standard Deviation

In [99]:
```
lst = [5,3,2,98,0,6,5,8383,937,25242,85,73,42]
lst.sort()
lst
```

Out[99]: [0, 2, 3, 5, 5, 6, 42, 73, 85, 98, 937, 8383, 25242]

In [100]:
```
np.mean(lst)
```

Out[100]: 2683.153846153846

In [101]:
```
arr1 = np.array(lst)
arr1
```

Out[101]:
```
array([    0,     2,     3,     5,     5,     6,    42,    73,    85,
          98,   937,  8383, 25242])
```

In [102]:
```
arr2 = arr1 - 2683.153846153846
arr2
```

Out[102]:
```
array([-2683.15384615, -2681.15384615, -2680.15384615, -2678.15384615,
       -2678.15384615, -2677.15384615, -2641.15384615, -2610.15384615,
       -2598.15384615, -2585.15384615, -1746.15384615,  5699.84615385,
       22558.84615385])
```

In [103]:
```
arr3 = arr2 ** 2
arr3
```

Out[103]:
```
array([7.19931456e+06, 7.18858595e+06, 7.18322464e+06, 7.17250802e+06,
       7.17250802e+06, 7.16715272e+06, 6.97569364e+06, 6.81290310e+06,
       6.75040341e+06, 6.68302041e+06, 3.04905325e+06, 3.24882462e+07,
       5.08901540e+08])
```

In [104]:
```
ad = sum(arr3)
ad
```

Out[104]:
```
614744153.6923077
```

In [106]:
```
var = ad/13
var
```

Out[106]:
```
47288011.82248521
```

In [107]:
```
np.sqrt(var)
```

Out[107]:
```
6876.627939803433
```

In [ ]:

In [108]:
```
np.sqrt(var) ** 2
```

Out[108]:
```
47288011.82248521
```

In [ ]:

In [ ]: